

GPS Software Architecture Document (SAD)

Revision: 01
Date: 16 / 06 / 2026
Author: illi Eisenberg

Table of Contents

1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions and Acronyms	6
1.4 References	6
2. System Overview	7
2.1 Driver Objectives	7
2.2 Supported Features	7
2.3 Supported GPS Protocols	8
2.4 Supported NMEA Sentences	8
2.5 Intended Usage	9
3. Architectural Goals	10
3.1 Protocol Independence	10
3.2 Modularity	10
3.3 Portability	10
3.4 Extensibility	10
3.5 Reusability	11
3.6 Maintainability	11
3.7 Efficiency	11
3.8 Reliability	12
4. High-Level Architecture	13
4.1 Architecture Overview	13
4.2 Architectural Layers	13
4.3 Component Diagram	15
4.4 Data Flow Overview	17
5. Driver Architecture	19
5.1 Architectural Overview	19
5.2 Public API Layer	19
5.3 Protocol Abstraction Layer	20
5.4 Protocol Implementation Layer	21

5.5 NMEA Processing Layer	22
5.6 UART Transport Layer	23
5.7 Reception Layer.....	23
5.8 Internal Data Storage.....	24
5.9 Design Principles	24
6. Runtime Behavior	25
6.1 Driver Initialization Sequence	25
6.2 Protocol Detection Sequence	25
6.3 Command Transmission Flow	25
6.4 NMEA Reception Flow.....	26
6.5 NMEA Parsing Flow.....	26
7. FreeRTOS Integration	27
7.1 Task Notifications.....	27
7.2 Queue Communication	27
7.3 UART Resource Protection.....	27
7.4 DMA Event Handling	28
8. Data Flow	28
8.1 Outgoing Commands	28
8.2 Incoming Sentences.....	29
9. Error Handling.....	30
9.1 Invalid Sentences	30
9.2 Checksum Failures.....	30
9.3 Unsupported Protocols.....	30
9.4 Communication Errors.....	30
10. Extensibility.....	31
10.1 Adding a New Protocol	31
10.2 Adding a New NMEA Sentence.....	31
10.3 Porting to Another MCU	31
11. Limitations.....	32
11.1 Current Supported Protocols	32
11.2 Current Supported Sentences	32
11.3 Assumptions.....	32
12. Future Enhancements.....	33
12.1 Additional GNSS Protocols.....	33

12.2 Additional NMEA Sentences	33
12.3 Event-Based Data Distribution	33
12.4 GPX Recording Support.....	33

1. Introduction

1.1 Purpose

This document describes the software architecture of the GPS Driver.

The purpose of this document is to define the overall structure, major software components, responsibilities, interfaces, and data flow of the driver. It provides a high-level architectural view intended to support development, maintenance, integration, and future extensions.

This document does not describe implementation details. Detailed algorithms, internal data structures, and function-level behavior are covered in separate design documents.

1.2 Scope

The GPS Driver provides a protocol-independent interface for controlling and communicating with GNSS receivers.

The driver supports command transmission, NMEA sentence reception, sentence routing, data parsing, and storage of parsed information. The architecture is designed to support multiple GPS vendor protocols through a common API.

The current implementation supports:

- PMTK protocol
- PCAS protocol
- NMEA sentence processing
- UART communication using DMA
- Ring-buffer based reception
- FreeRTOS integration

The driver is intended for embedded systems and has been developed and tested on STM32 microcontrollers. The architecture is designed to allow adaptation to other microcontroller platforms with minimal modifications.

1.3 Definitions and Acronyms

Term	Description
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
NMEA	National Marine Electronics Association sentence format
PMTK	MediaTek proprietary GPS command protocol
PCAS	CASIC proprietary GPS command protocol
DMA	Direct Memory Access
UART	Universal Asynchronous Receiver/Transmitter
API	Application Programming Interface
ISR	Interrupt Service Routine
RMC	Recommended Minimum Navigation Information
GSV	Satellites in View
GSA	GNSS DOP and Active Satellites
TXT	Text Information Message

1.4 References

The following documents were used during the development of the driver:

- CASIC Multimode Satellite Navigation Receiver Protocol Specification
- PMTK Command Packet User Manual
- NMEA 0183 Protocol Specification
- STM32 Reference Manuals
- FreeRTOS Documentation

2. System Overview

2.1 Driver Objectives

The GPS Driver provides a unified and protocol-independent interface for GNSS receivers.

Its primary objectives are:

- Simplify integration of GPS modules into embedded applications.
- Support multiple vendor-specific command protocols through a common API.
- Provide reliable reception and processing of NMEA sentences.
- Isolate protocol-specific implementation details from application code.
- Support future expansion with additional GPS protocols and NMEA sentence types.
- Minimize application-level dependency on GPS hardware details.

2.2 Supported Features

The driver currently provides the following functionality:

2.2.1 Device Control

- GPS initialization
- Protocol selection
- Protocol detection
- Baud-rate configuration
- Hot start
- Cold start
- Restart
- Standby mode
- Firmware version request

2.2.2 NMEA Output Control

- Enable RMC output
- Disable RMC output
- Enable GSV output
- Disable GSV output

2.2.3 NMEA Processing

- Sentence validation
- Checksum verification
- Sentence routing
- Sentence parsing

2.2.4 Data Extraction

- Position information
- Date and time information
- Speed information
- Course information
- Satellite information
- Firmware information

2.2.5 Communication

- UART transmission
- DMA-based UART reception
- Ring-buffer management

2.3 Supported GPS Protocols

The architecture supports multiple GPS command protocols.

Currently implemented protocols are:

Protocol	Description
PMTK	MediaTek proprietary command protocol
PCAS	CASIC proprietary command protocol

The protocol layer is isolated from the application layer through a common function interface, allowing additional protocols to be added with minimal changes to the existing architecture.

2.4 Supported NMEA Sentences

The driver currently supports parsing of the following NMEA sentences:

Sentence Description	
RMC	Recommended Minimum Navigation Information
GSV	Satellites in View
GSA	GNSS DOP and Active Satellites
TXT	Firmware and module information

Additional NMEA sentence types can be integrated through the routing and parser framework.

2.5 Intended Usage

The driver is intended for embedded systems requiring GNSS functionality, including:

- Asset tracking systems
- LoRa-based positioning networks
- Telemetry devices
- Environmental monitoring stations
- Vehicle tracking systems
- Portable battery-powered devices

The architecture supports both periodic data acquisition and event-driven operation.

3. Architectural Goals

3.1 Protocol Independence

The driver shall provide a unified interface that is independent of the underlying GPS protocol.

Application code shall interact only with the public GPS API and shall not require knowledge of protocol-specific commands or message formats.

Protocol-specific functionality shall be encapsulated within dedicated protocol modules.

This approach simplifies integration and allows protocol replacement without affecting higher software layers.

3.2 Modularity

The architecture shall be organized into independent software modules, each responsible for a specific function.

Major responsibilities are separated into distinct layers, including:

- GPS core management
- Protocol handling
- NMEA processing
- UART communication
- Data storage

This separation reduces coupling between components and improves maintainability.

3.3 Portability

The driver shall minimize hardware-specific dependencies.

Hardware-dependent functionality shall be isolated behind dedicated interfaces, allowing the driver to be adapted to different microcontrollers, operating systems, or communication layers with minimal modifications.

Although the current implementation targets STM32 microcontrollers and FreeRTOS, the architecture is intended to remain largely platform independent.

3.4 Extensibility

The architecture shall support future expansion without requiring significant modification of existing components.

Examples of future extensions include:

- Additional GPS protocols
- Additional NMEA sentence types
- Alternative communication interfaces
- New GNSS receiver families

New functionality should be integrated through existing architectural interfaces whenever possible.

3.5 Reusability

The driver shall be reusable across multiple projects and applications.

Application-specific logic shall remain outside the driver boundary.

The driver shall focus exclusively on GPS communication, command handling, sentence processing, and data extraction.

Functions related to user interfaces, networking, data logging, or application workflows are intentionally excluded from the driver architecture.

3.6 Maintainability

The architecture shall promote readability, maintainability, and long-term support.

Responsibilities shall be clearly defined and distributed among software modules.

Changes made within one module should have minimal impact on other modules.

This approach reduces development effort during debugging, feature additions, and future refactoring activities.

3.7 Efficiency

The architecture shall support operation in resource-constrained embedded environments.

Design decisions should minimize:

- CPU utilization
- RAM consumption
- Communication overhead

Efficient processing of incoming NMEA data is achieved through DMA-based UART reception and buffered sentence handling.

3.8 Reliability

The driver shall provide reliable reception and processing of GPS information.

Incoming NMEA sentences shall undergo:

- Structure validation
- Checksum verification
- Sentence identification
- Controlled routing to dedicated parsers

Invalid or corrupted data shall be rejected before reaching application-visible data structures.

4. High-Level Architecture

4.1 Architecture Overview

The GPS Driver is organized as a layered architecture that separates protocol handling, communication, sentence processing, and data management into independent software components.

The application interacts only with the public GPS API. All protocol-specific details, communication mechanisms, and parsing logic are hidden behind dedicated layers.

This approach improves maintainability, portability, and extensibility while reducing coupling between application code and GPS-specific implementation details.

4.2 Architectural Layers

The driver consists of the following logical layers:

4.2.1 Application Layer

The application layer represents the software components that use GPS services.

Typical responsibilities include:

- Requesting GPS initialization
- Enabling or disabling NMEA sentence output
- Requesting firmware information
- Retrieving parsed GPS data
- Processing GPS information for higher-level functions

The application does not directly communicate with protocol-specific modules.

4.2.2 GPS Core Layer

The GPS Core Layer provides the public API exposed to the application.

Responsibilities include:

- Driver initialization
- Protocol selection
- Context management
- Command dispatching
- Access to parsed GPS information

This layer acts as the central coordination point for all GPS-related operations.

4.2.3 Protocol Layer

The Protocol Layer contains protocol-specific implementations.

Currently supported protocols:

- PMTK
- PCAS

Responsibilities include:

- Command generation
- Protocol-specific configuration
- Device control commands
- Protocol abstraction

Each protocol implementation is isolated from the application and accessed through a common interface.

4.2.4 NMEA Processing Layer

The NMEA Processing Layer is responsible for processing incoming GPS messages.

Responsibilities include:

- Sentence validation
- Checksum verification
- Sentence identification
- Sentence routing
- Data extraction

The layer provides protocol-independent parsing of NMEA sentences.

4.2.5 Communication Layer

The Communication Layer provides access to the physical communication channel.

Responsibilities include:

- UART transmission
- DMA-based reception
- Ring-buffer management
- Reception event generation

This layer isolates hardware communication from higher software layers.

4.2.6 Data Storage Layer

The Data Storage Layer stores parsed information extracted from GPS messages.

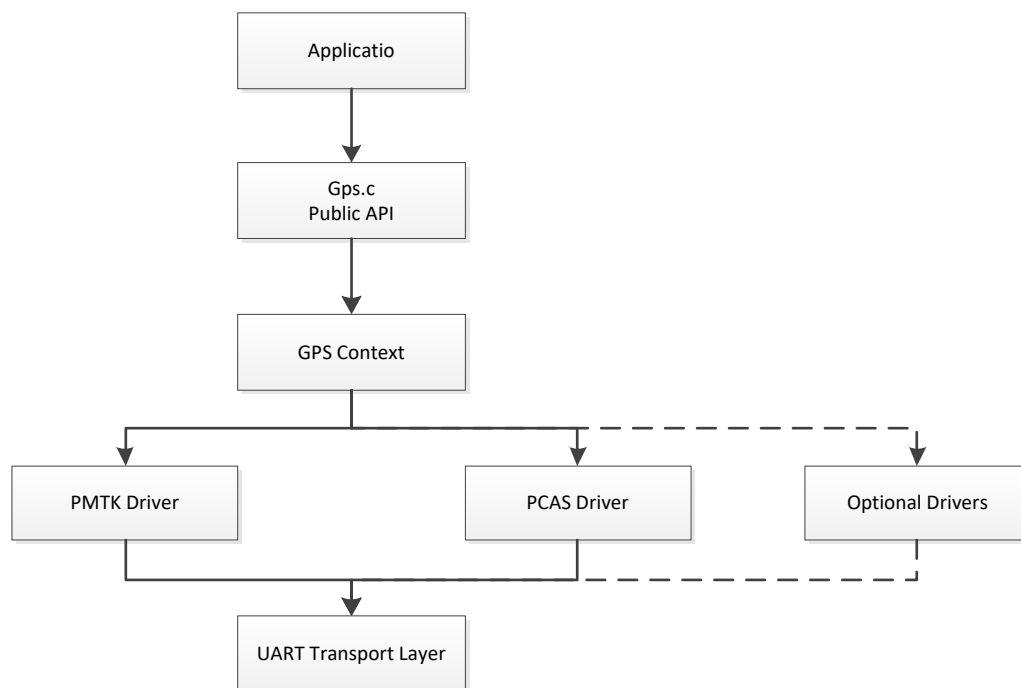
Examples include:

- Position information
- Date and time
- Satellite information
- Firmware information
- Receiver status

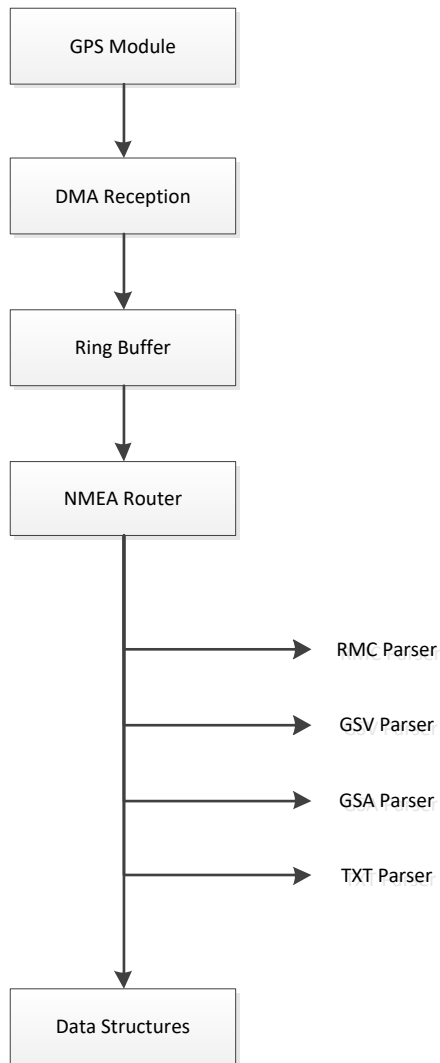
The stored data is made available to the application through the GPS API.

4.3 Component Diagram

The following diagram illustrates the relationship between major software components.



Incoming Data Path:



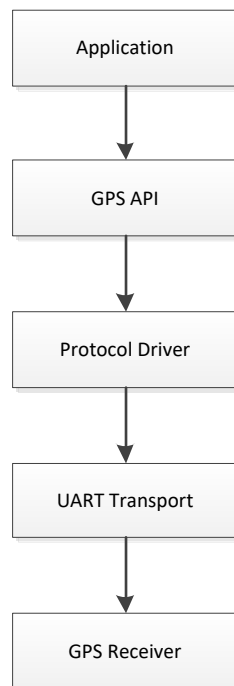
4.4 Data Flow Overview

The driver supports two primary data flows:

4.4.1 Outgoing Command Flow

The application issues a request through the public API.

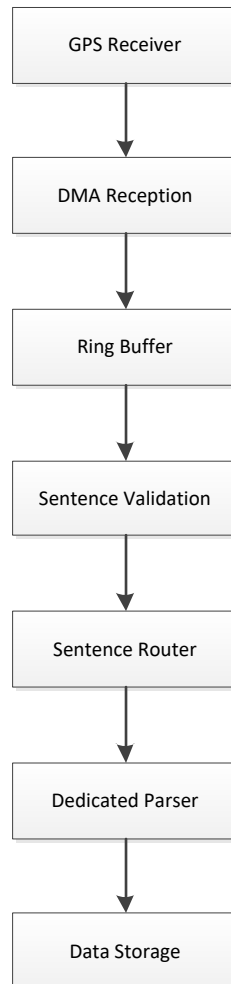
The request is forwarded to the active protocol implementation, translated into a protocol-specific command, and transmitted to the GPS receiver through the communication layer.



4.4.2 Incoming Data Flow

GPS messages are received through the UART interface using DMA.

The received data is accumulated inside a ring buffer. Complete NMEA sentences are extracted, validated, routed to the appropriate parser, and stored in internal data structures.



5. Driver Architecture

The GPS driver is organized into a set of independent modules, each responsible for a specific aspect of the GPS subsystem. This separation simplifies maintenance, improves portability, and allows individual components to be reused in other projects.

5.1 Architectural Overview

The driver follows a layered architecture:

Application Layer	
GPS Public API (gps.c / gps.h)	
Protocol Abstraction Layer Function Pointer Interface	
PMTK Driver (gps_pmtkN.c/.h)	PCAS Driver gps_pcasN.c/.h)
NMEA Parser Layer (gps_nmea.c / .h)	
UART Transport Layer (uart_transport.c / .h)	
Ring Buffer + DMA Reception Layer (uartRingBufDMA.c / .h)	
Hardware Layer UART + DMA + GPS	

5.2 Public API Layer

Files:

gps.c
gps.h

Responsibilities:

- Driver initialization
- Protocol detection
- GPS command dispatching
- Public API exposed to the application
- Management of the global GPS context

Examples:

```
GPS_Init();  
GPS_StartRMC();  
GPS_StopRMC();  
GPS_StartGSV();  
GPS_Set_Baud();  
GPS_GetVersion();
```

The application communicates only with this layer and remains independent of the underlying GPS protocol.

5.3 Protocol Abstraction Layer

Files:

```
gps_types.h  
gps_internal.h
```

Responsibilities:

- Definition of the GPS context structure
- Function pointer interface
- Runtime protocol binding

Example:

```
ctx->start_rmc  
ctx->stop_rmc  
ctx->get_version  
ctx->set_baud
```

During initialization the driver detects the connected GPS module and binds the proper protocol implementation.

Example:

```
GPS_PMTK_Bind(&gps_ctx);
```

or

```
GPS_PCAS_Bind(&gps_ctx);
```

This mechanism allows support for multiple GPS vendors without modifying the application code.

5.4 Protocol Implementation Layer

Files:

```
gps_pmtk.c  
gps_pmtk.h
```

```
gps_pcas.c  
gps_pcas.h
```

Responsibilities:

- Protocol-specific command generation
- Command formatting
- Checksum generation
- GPS configuration

Examples:

PMTK

```
PMTK314  
PMTK251  
PMTK605
```

PCAS

```
PCAS03  
PCAS01  
PCAS06  
PCAS10
```

Each protocol module converts generic API requests into vendor-specific commands.

For example:

```
GPS_StartRMC();
```

may generate either:

```
$PMTK314,...
```

or

```
$PCAS03,...
```

depending on the detected GPS module.

5.5 NMEA Processing Layer

Files:

gps_nmea.c
gps_nmea.h

Responsibilities:

- Sentence validation
- Checksum verification
- Sentence routing
- NMEA parsing

Supported message types currently include:

RMC
GSA
GSV
TXT (for PCAS specific response)

The parser extracts information from incoming NMEA sentences and stores it in internal driver structures.

Examples of parsed data:

- Position
- Speed
- Course
- Date and time
- Satellite information
- Firmware information
- Fix status

This layer is independent from the protocol command layer and can process NMEA messages regardless of the GPS vendor.

5.6 UART Transport Layer

Files:

```
uart_transport.c  
uart_transport.h
```

Responsibilities:

- UART transmission abstraction
- Hardware-independent interface
- Centralized transmit handling

Example:

```
UART_Send(...)
```

The protocol modules never access the hardware directly and instead use this transport layer.

5.7 Reception Layer

Files:

```
uartRingBufDMA.c  
uartRingBufDMA.h
```

Responsibilities:

- Continuous DMA reception
- Ring buffer management
- Detection of complete NMEA sentences
- Notification of higher software layers

The implementation is based on a ring buffer originally published by *Controllerstech* and later adapted for this project.

The reception layer operates independently of the parser and ensures that incoming GPS data is captured without blocking the CPU.

5.8 Internal Data Storage

Files:

```
gps_data.c  
gps_data.h
```

Responsibilities:

- Storage of parsed GPS information
- Storage of satellite information
- Storage of firmware information

Examples:

```
GPSTData_t  
GPSSats_t  
GPSfw_t
```

These structures serve as the internal data repository for the driver.

The application retrieves information through dedicated APIs rather than accessing the parser directly.

5.9 Design Principles

The driver architecture was designed according to the following principles:

- Separation of responsibilities
- Protocol independence
- Reusability
- Low coupling between modules
- Support for multiple GPS vendors
- Non-blocking reception using DMA
- Compatibility with RTOS-based systems
- Adaptability for bare-metal environments

The architecture allows future extensions such as:

- Additional GPS protocols
- Binary protocol support
- Logging subsystems
- GPX recording services
- Navigation and tracking features
- Distance and bearing calculations between GPS coordinates

6. Runtime Behavior

6.1 Driver Initialization Sequence

The GPS driver initialization sequence prepares the communication infrastructure and establishes connectivity with the GPS receiver.

During system startup, the driver initializes the UART transport layer, enables DMA-based reception, initializes the ring buffer subsystem, and performs protocol detection. Once a supported protocol is identified, the corresponding protocol implementation is bound to the driver.

After successful initialization, the GPS driver enters its operational state and becomes available to the application layer.

6.2 Protocol Detection Sequence

The driver supports multiple GPS protocols through a protocol abstraction layer.

During initialization, the driver attempts to identify the protocol used by the connected GPS receiver. Detection is performed by transmitting protocol-specific commands and evaluating the received responses.

When a valid response is detected, the matching protocol implementation is selected and bound to the driver context. If no supported protocol is detected, the initialization process fails.

This mechanism allows the application layer to remain independent of protocol-specific details.

6.3 Command Transmission Flow

GPS commands originate from the application layer and are routed through the driver abstraction layer.

The driver translates generic GPS operations into protocol-specific commands and forwards them to the GPS receiver through the UART transport layer.

Examples of supported operations include:

- Version requests
- Receiver restart
- Standby mode control
- Baud rate configuration
- NMEA sentence enable and disable requests

The application layer is isolated from protocol-specific command formats.

6.4 NMEA Reception Flow

Incoming NMEA data is continuously received through the UART interface using DMA.

Received characters are stored in the ring buffer subsystem until a complete NMEA sentence is detected. Once a complete sentence becomes available, the driver notifies the processing layer that new data is ready for evaluation.

This architecture separates data reception from data processing and minimizes CPU utilization during continuous GPS operation.

6.5 NMEA Parsing Flow

After a complete NMEA sentence is extracted from the reception buffer, the driver performs validation and classification.

Valid sentences are routed to the appropriate parser according to their sentence type. The parser extracts relevant information and updates the internal GPS data repository.

Examples of supported sentence types include:

- RMC (Position, Time, Speed)
- GSV (Satellite Information)
- GSA (Fix Information)
- TXT (Receiver Information)

The parsed information becomes available to higher software layers through the driver data model.

7. FreeRTOS Integration

7.1 Task Notifications

The GPS driver utilizes FreeRTOS task notifications to signal asynchronous events between execution contexts.

Task notifications are used to inform higher software layers when GPS-related activities require processing, such as the arrival of newly received NMEA data or the completion of command-related operations.

This mechanism provides a lightweight and efficient alternative to more complex synchronization primitives.

7.2 Queue Communication

The GPS subsystem exchanges information with other software components through FreeRTOS queues.

Queues are used to transfer GPS-related data, commands, and application messages between tasks while maintaining execution context isolation.

This approach supports modular software architecture and prevents direct dependencies between the GPS driver and higher application layers.

7.3 UART Resource Protection

The UART interface is considered a shared system resource and is protected to prevent concurrent access from multiple execution contexts.

Access synchronization ensures that command transmissions and data reception operations do not interfere with each other.

This mechanism improves communication reliability and prevents data corruption caused by simultaneous UART usage.

7.4 DMA Event Handling

The driver utilizes DMA-based UART reception to minimize CPU utilization during continuous GPS operation.

DMA events are used to detect newly received data and trigger the processing flow responsible for sentence extraction and parsing.

This architecture allows efficient handling of continuous NMEA streams while maintaining low processor overhead and deterministic task execution.

8. Data Flow

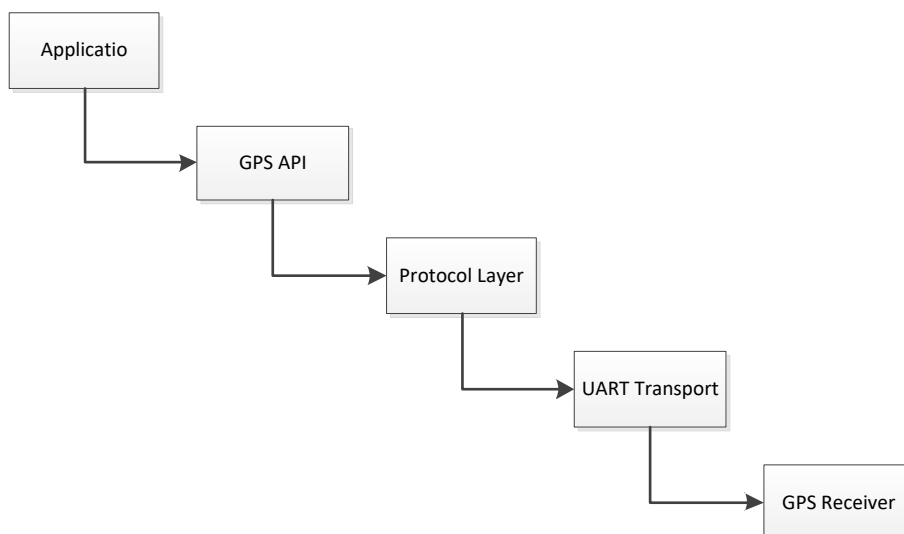
8.1 Outgoing Commands

Outgoing GPS commands originate from the application layer and are issued through the GPS public interface.

The GPS API provides a protocol-independent abstraction that allows the application to request GPS operations without knowledge of the underlying receiver protocol.

The protocol layer translates generic GPS operations into protocol-specific command formats and forwards them to the UART transport layer for transmission to the GPS receiver.

This architecture separates application logic from protocol implementation details and enables support for multiple GPS protocols within a unified framework.



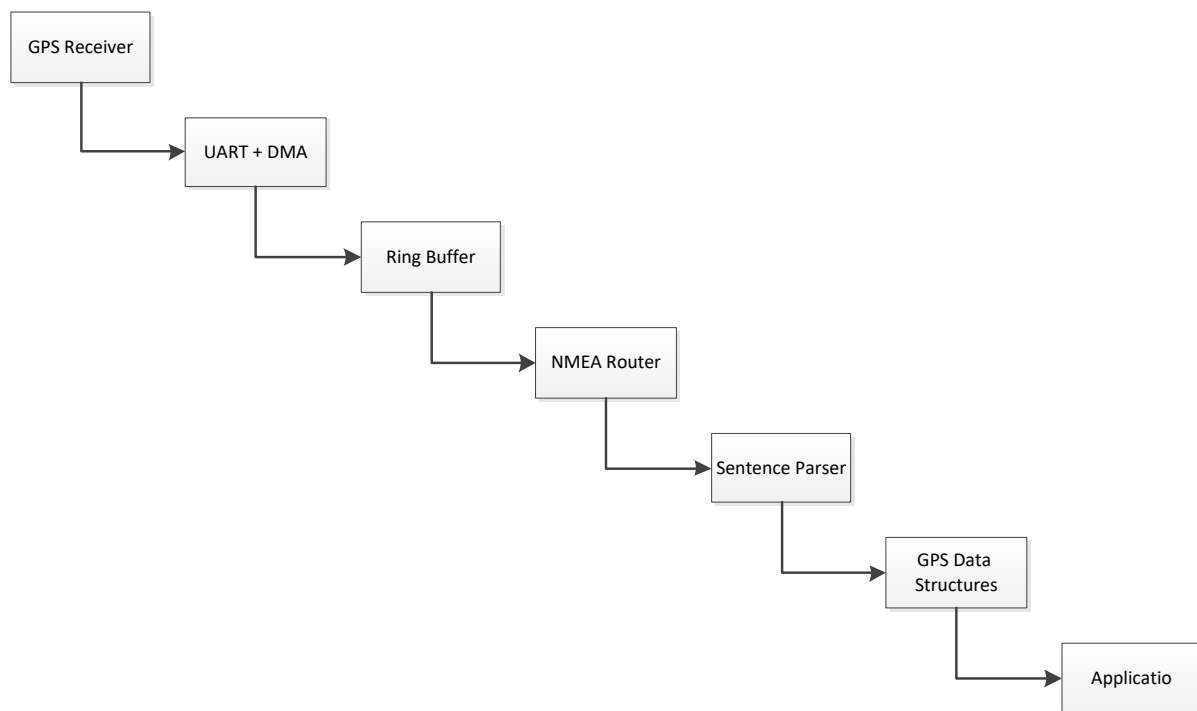
8.2 Incoming Sentences

Incoming NMEA data is continuously received from the GPS receiver through the UART interface.

DMA-based reception transfers the incoming data stream into memory with minimal CPU involvement. The received data is accumulated within the ring buffer subsystem until complete NMEA sentences become available.

The NMEA routing layer identifies the sentence type and forwards the sentence to the appropriate parser. Each parser extracts the relevant information and updates the corresponding internal data structures.

The resulting GPS information becomes available to higher software layers through a consistent and protocol-independent data model.



9. Error Handling

9.1 Invalid Sentences

The GPS driver validates incoming NMEA sentences before processing.

Sentences that do not conform to the expected NMEA structure are rejected and excluded from further processing. This prevents malformed or incomplete data from propagating into the internal GPS data model.

The driver continues normal operation and remains capable of processing subsequent valid sentences.

9.2 Checksum Failures

Checksum verification is performed on incoming NMEA sentences to ensure data integrity.

If a checksum validation failure is detected, the sentence is discarded, and no internal data is updated based on the received information.

This mechanism protects the system from communication errors, corrupted data, and incomplete message reception.

9.3 Unsupported Protocols

The driver supports a predefined set of GPS receiver protocols through a protocol abstraction layer. If protocol detection does not identify a supported receiver protocol, the driver remains uninitialized, and GPS services are not made available to higher software layers.

This behavior prevents undefined operation when communicating with unsupported GPS devices.

9.4 Communication Errors

Communication errors may occur due to UART failures, cabling issues, receiver malfunction, or loss of signal transmission. The driver is designed to tolerate temporary communication interruptions and continue operation when communication is restored.

Higher software layers may use the availability and validity of received GPS data to determine the current operational status of the receiver.

10. Extensibility

10.1 Adding a New Protocol

The GPS driver is designed around a protocol abstraction layer that separates protocol-specific behavior from the core driver functionality.

New GPS protocols can be integrated by implementing the required protocol-specific operations and connecting them to the existing driver framework.

This architecture allows the system to support additional GPS receivers without impacting application-level software or existing protocol implementations.

10.2 Adding a New NMEA Sentence

The NMEA processing architecture is organized around sentence classification and routing.

New NMEA sentence types can be incorporated by extending the sentence routing mechanism and providing a corresponding parser implementation.

This approach enables incremental support for additional GPS information while preserving compatibility with existing functionality.

10.3 Porting to Another MCU

The driver architecture isolates hardware-dependent functionality from protocol and parsing logic.

Platform-specific components such as UART communication, DMA configuration, operating system integration, and low-level hardware services can be adapted independently of the core GPS functionality.

This separation simplifies migration to different microcontrollers, development boards, or software platforms while preserving the overall driver architecture.

11. Limitations

11.1 Current Supported Protocols

The current implementation supports a limited set of GPS receiver protocols.

Protocol support is focused on the protocols required by the target GPS receivers used during development and validation.

Additional protocols can be integrated through the existing protocol abstraction architecture, but are not included in the current release.

11.2 Current Supported Sentences

The driver currently processes a subset of standard and vendor-specific NMEA sentences.

Supported sentences provide the information required for the intended application domains, including position, navigation status, satellite information, and receiver identification data.

NMEA sentences that are not explicitly supported are ignored by the processing layer and do not affect normal driver operation.

11.3 Assumptions

The architecture assumes that the GPS receiver communicates through a UART interface and produces NMEA-compatible output.

The driver also assumes that the underlying platform provides the required communication services and timing infrastructure necessary for reliable operation.

Correct operation depends on a valid GPS receiver configuration, stable serial communication, and availability of GPS signal reception.

12. Future Enhancements

12.1 Additional GNSS Protocols

The architecture is designed to support the integration of additional GNSS receiver protocols beyond those currently implemented. Future protocol support may include receivers from other vendors or emerging protocol standards while preserving the existing protocol abstraction model. This capability enables long-term adaptability to evolving GNSS hardware platforms.

12.2 Additional NMEA Sentences

Future releases may expand support for additional NMEA sentence types to provide access to a broader range of navigation and receiver information. The existing sentence routing architecture allows new sentence types to be incorporated without a significant impact on the overall system structure. This enhancement can increase the amount of available GNSS information while maintaining architectural consistency.

12.3 Event-Based Data Distribution

Future versions may introduce a fully event-driven data distribution mechanism for GPS information. Instead of relying solely on periodic data retrieval, GPS updates could be propagated automatically when new information becomes available.

Such an approach may improve responsiveness, reduce unnecessary processing, and simplify integration with higher-level software components.

12.4 GPX Recording Support

Future enhancements may include support for GPX track recording and route logging. This capability would allow GPS position data to be stored for later analysis, visualization, or export to external mapping applications.

The existing architecture provides a suitable foundation for integrating location recording functionality without requiring major architectural changes.